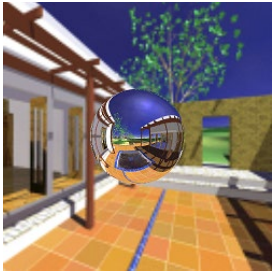


Worksheet 7: Environment mapping and normal mapping

Reading	Angel: Sections 7.5.6 and 7.7-7.9
Purpose	The purpose of this set of exercises is to become familiar with the concepts behind environment mapping and normal mapping. We will use environment mapping to render a curved reflector and, in the process, learn how to use cube maps and the reflection function. We will also use normal mapping to add scale-like surface details.
Part 1 Cube map 	Start from a textured sphere (Part 3 of Worksheet 6). Instead of the ordinary 2D texture, we will now use a cube map to texture the sphere. [Angel 7.8] Modify your texture initialization such that it loads a cube map from six image files, one file for each face. The files are in textures.zip (on DTU Learn). The file names and their orientation in the cube map are: <pre>var cubemap = ['textures/cm_left.png', // POSITIVE_X 'textures/cm_right.png', // NEGATIVE_X 'textures/cm_top.png', // POSITIVE_Y 'textures/cm_bottom.png', // NEGATIVE_Y 'textures/cm_back.png', // POSITIVE_Z 'textures/cm_front.png']; // NEGATIVE_Z</pre> If you use a loop to load the image files, you must retrieve the image from the event variable in the onload function: <pre>image.onload = function(event) { var image = event.target; // Insert cube face texture initialization here };</pre> Once the cube map is initialized, no inverse map is needed to compute texture coordinates. Simply use the world space normal as texture coordinates when looking up the texture color in the fragment shader.
Part 2 Environment 	The next step is to also draw the environment in the background. To do this, we draw a screen-filling quad very close to the far plane of the view frustum and texture it using the cube map. - A screen-filling quad close to the far plane is most easily drawn using clip coordinates, where the diagonal goes from $(-1, -1, 0.999, 1)$ to $(1, 1, 0.999, 1)$. Insert this background quad into your scene. - Draw the background quad using the same shaders as in Part 1 but introduce a uniform matrix M_{tex} in the vertex shader that transforms the vertex position to texture coordinates. - For the sphere, M_{tex} is an identity matrix. The vertices of the background quad are however in clip space, so its model-view-projection matrix is an identity matrix, but its M_{tex} should transform from clip space positions to world space directions. Create M_{tex} for the background quad using (a) the inverse of the projection matrix to go from clip coordinates to camera coordinates and (b) the inverse of the rotational part of the view matrix (no translation) to get direction vectors in world coordinates. Explain the transformation.

Worksheet 7: Environment mapping and normal mapping

Part 2 Reflection 	The sphere is not really like a mirror ball. Instead of looking up the environment in the normal direction, we should look up the environment in the direction of reflection. [Angel 7.7] • Create a uniform variable (<code>reflective</code>) to distinguish reflective objects (the mirror ball) from other objects (the background quad). • Upload the eye position to the fragment shader as a uniform variable and compute the direction of incidence (the view vector, $\mathbf{v}$) in world space coordinates. • Use a conditional operator (or an <code>if</code>-statement) to choose the direction of reflection as texture coordinates for reflective objects. Hint: You can use the built-in GLSL ES function <code>reflect(vec3 incident, vec3 normal)</code>.
Part 4 Bump mapping  	Finally, we will perturb the normal of the mirror ball using a normal map to give the impression that the ball surface is 'bumpy'. [Angel 7.9] • Load the normal map texture from the file <code>textures/normalmap.png</code>. Map it onto the sphere using the same technique as in Part 3 of Worksheet 6. The first image for this part is the expected result. • Bind the normal map to another texture (e.g. <code>TEXTURE1</code>) so that it can be used together with the cube map [Angel 7.5.6]. The color found in the normal map is in $[0, 1]$. Transform it to be in $[-1, 1]$ to get the actual normal. • The normal retrieved from the normal map is in tangent space. We need to transform it to world space to use it in place of the sphere normal when calculating the direction of reflection. Use the following function as an efficient way to perform this change of basis transformation:¹ <pre>vec3 rotate_to_normal(vec3 n, vec3 v) { float sgn_nz = sign(n.z + 1.0e-16); float a = -1.0/(1.0 + abs(n.z)); float b = n.x*n.y*a; return vec3(1.0 + n.x*n.x*a, b, -sgn_nz*n.x)*v.x + vec3(sgn_nz*b, sgn_nz*(1.0 + n.y*n.y*a), -n.y)*v.y + n*v.z; }</pre> The first argument is the surface normal $\mathbf{n}$ in world coordinates, the second argument is the tangent space vector to be transformed to world space. In our case, the tangent space vector is the normal retrieved from the normal map. The transformation returns the bump mapped normal to be used in place of the sphere normal when rendering reflective objects.

¹ Frisvad, J. R. Building an orthonormal basis from a 3D unit vector without normalization. *Journal of Graphics Tools* 16(3), pp. 151-159, August 2012. <https://people.compute.dtu.dk/jerf/code/hairy/>

Duff, T., Burgess, J., Christensen, P., Hery, C., Kensler, A., Liani, M., and Villemin, R. Building an orthonormal basis, revisited. *Journal of Computer Graphics Techniques* 6(1), pp. 1-8. March 2017.